**МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ**
**НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ**
**«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»**
**НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ АТОМНОЇ ТА ТЕПЛОВОЇ ЕНЕРГЕТИКИ**
**КАФЕДРА ІНЖЕНЕРІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ В ЕНЕРГЕТИЦІ**

# КУРСОВА РОБОТА

з дисципліни: «Основи Веб-програмування. Курсова робота»

**на тему: «Додаток для новин NewsHub»**

**Керівник: Виконав:**
Недашківський О.Л. Слюсар Назарій Олегович студент 2 курсу
«Допущено до захисту» групи ТВ-з41

«___» _____ 20__ р. _____ Захищено з оцінкою
(особистий підпис виконавця) _____ «___» _____20__ р.
(оцінка)

Посилання на GitHub репозиторій https://github.com/NazariiSliusar/Kursova_web.git

КИЇВ – 2026

# АНОТАЦІЯ

Тема курсової роботи: «Додаток для новин NewsHub». Метою роботи є розробка веб-додатку для перегляду, пошуку та збереження новин з підтримкою двох мов і двох тем оформлення.

Серверна частина реалізована на Node.js з використанням Express. Дані зберігаються у SQLite через бібліотеку sql.js. Клієнтська частина написана на vanilla JavaScript без фреймворків. Для отримання новин інтегровані сервіси NewsAPI та GNews.

Обсяг пояснювальної записки 22 сторінки, кількість ілюстрацій 5, використаних джерел 10.

# ANNOTATION

The topic of the course work: "NewsHub News Application". The purpose of the work is to develop a web application for browsing, searching and saving news with support for two languages and two visual themes.

The server side is implemented in Node.js using Express. Data is stored in SQLite via the sql.js library. The client side is written in vanilla JavaScript without frameworks. NewsAPI and GNews services are integrated for fetching news.

The explanatory note is 22 pages long, the number of illustrations is 5, and the number of references is 10.

**ЗМІСТ**

АНОТАЦІЯ

ВСТУП

РОЗДІЛ 1. ДІАГРАМИ КОМПОНЕНТІВ, ВЗАЄМОДІЇ ТА КЛАСІВ

РОЗДІЛ 2. СТРУКТУРА МЕНЮ

РОЗДІЛ 3. РОЗМІЩЕННЯ ТА СТИЛІЗАЦІЯ ІНТЕРФЕЙСУ

РОЗДІЛ 4. МОДЕЛІ ДАНИХ

РОЗДІЛ 5. МЕХАНІЗМИ ОТРИМАННЯ ТА ОНОВЛЕННЯ ДАНИХ

РОЗДІЛ 6. БАЗА ДАНИХ

РОЗДІЛ 7. ІНТЕГРАЦІЯ СТОРОННІХ СЕРВІСІВ

РОЗДІЛ 8. СЕРВЕРНА АРХІТЕКТУРА

РОЗДІЛ 9. ОБРОБКА ЗАПИТІВ ТА ВЗАЄМОДІЯ З БАЗОЮ ДАНИХ

РОЗДІЛ 10. ТЕСТУВАННЯ ТА НАЛАШТУВАННЯ

ВИГЛЯД СТОРІНОК ВЕБ-САЙТУ

ВИСНОВКИ

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

# ВСТУП

У сучасному світі кількість новинних джерел постійно зростає, і користувачам стає дедалі важче стежити за актуальними подіями, відвідуючи кожен сайт окремо. Агрегатори новин вирішують цю проблему, збираючи матеріали з різних джерел в одному місці. При цьому більшість популярних рішень, таких як Google News або Feedly, або не підтримують українську мову, або обмежують функціонал у безкоштовній версії.

Метою курсової роботи є розробка веб-додатку «NewsHub» новинного агрегатора з підтримкою двох мов (англійської та української) і двох тем оформлення, можливістю пошуку, фільтрації за категоріями, збереження закладок та ведення історії пошуку.

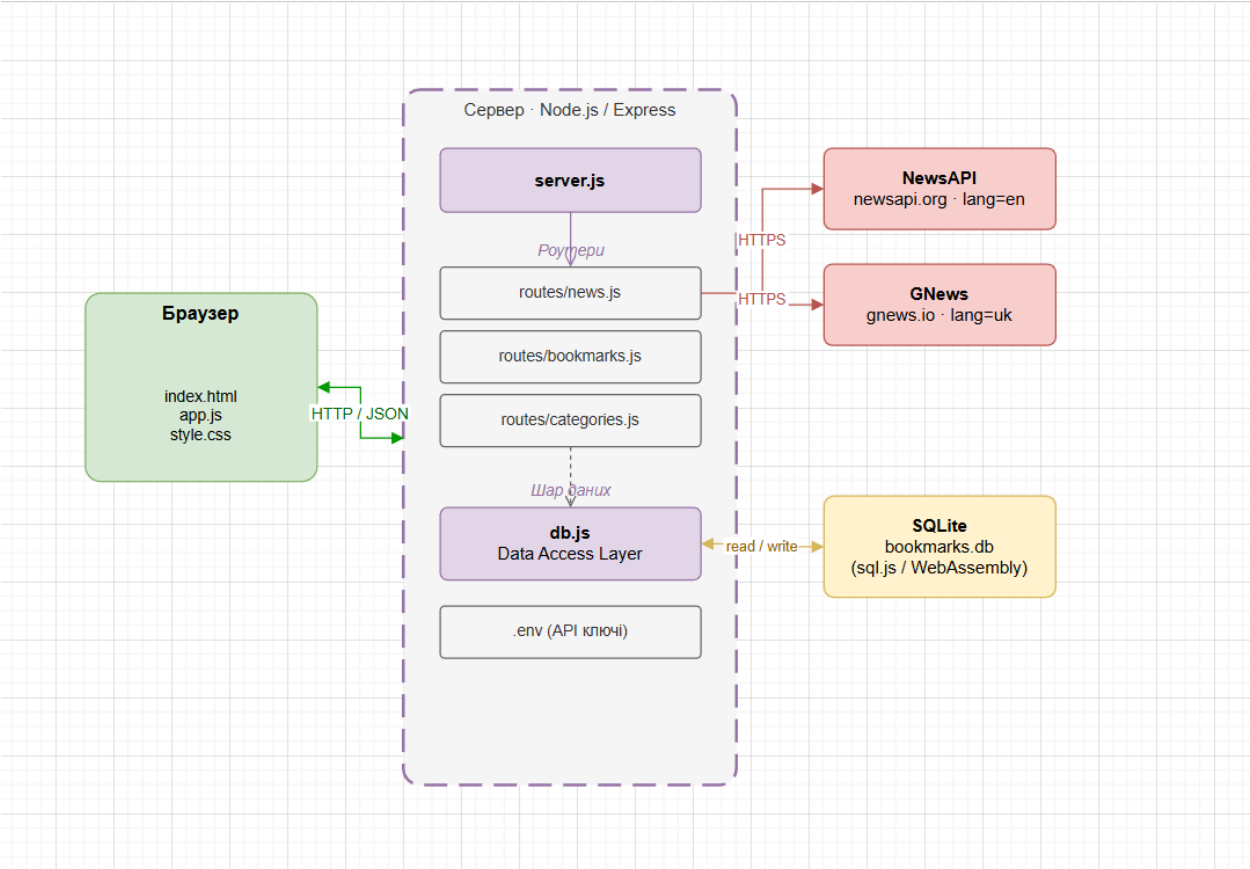
У ході роботи вирішено наступні завдання:

- спроєктовано діаграми компонентів, взаємодії та класів;
- розроблено структуру меню та інтерфейс застосунку;
- реалізовано серверний REST API на Node.js / Express;
- спроєктовано та реалізовано базу даних SQLite через sql.js;
- інтегровано зовнішні сервіси NewsAPI та GNews;
- написано базові автотести для перевірки API та бази даних.

# РОЗДІЛ 1. ДІАГРАМИ КОМПОНЕНТІВ, ВЗАЄМОДІЇ ТА КЛАСІВ

На першому етапі роботи було спроєктовано структуру майбутнього додатку у вигляді UML-діаграм. Діаграми зберігаються у теці /docs репозиторію.

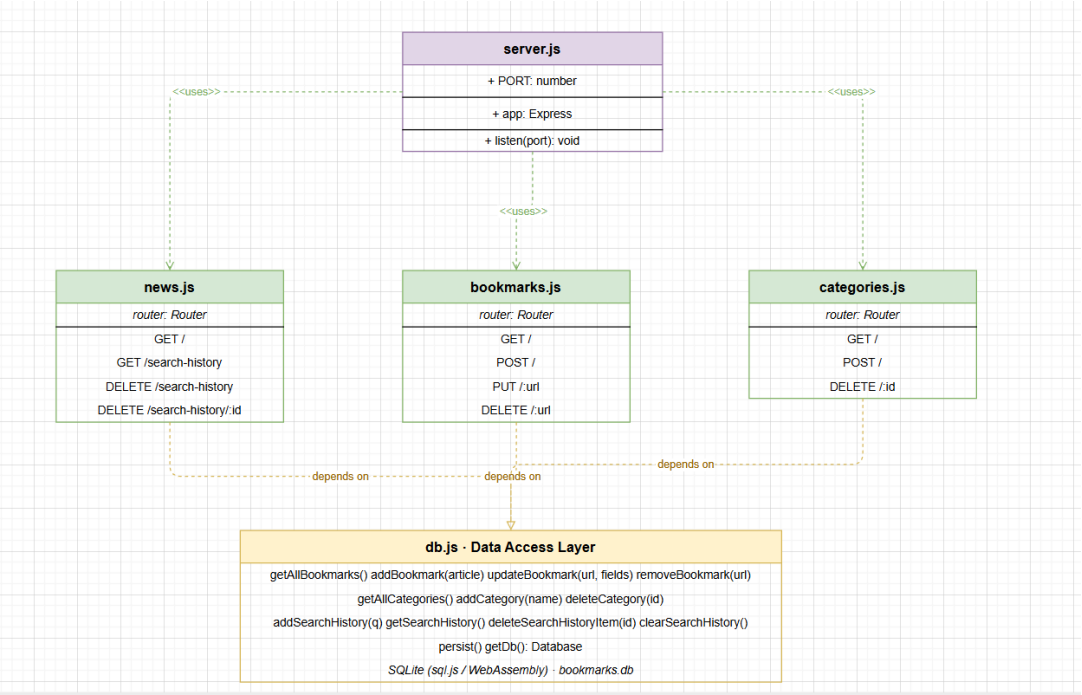
Діаграма компонентів показує три основні блоки системи: клієнтський браузер (HTML + JS + CSS), серверний застосунок (Node.js / Express) та зовнішні сервіси (NewsAPI, GNews і файлова база даних SQLite). Клієнт взаємодіє лише з сервером через HTTP; прямого зв'язку між браузером і зовнішніми API немає це важливо з точки зору безпеки ключів.




[Рисунок 1.1 – Діаграма компонентів додатку NewsHub]

Діаграма взаємодії (sequence diagram) описує типовий сценарій — запит новин: користувач вводить пошуковий запит, клієнт відправляє GET /api/news?q=...&lang=en, сервер формує URL до NewsAPI або GNews залежно

від параметру lang, отримує відповідь, нормалізує формат і повертає масив статей. Паралельно запит автоматично зберігається в таблицю search_history.




[Рисунок 1.2– Діаграма класів додатку NewsHub]

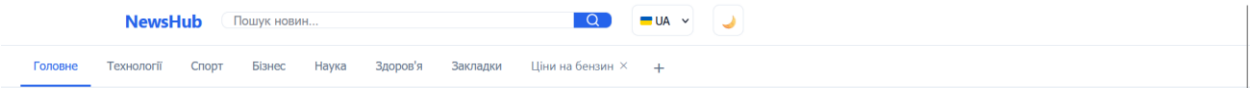
Діаграма класів відображає структуру серверного шару: модуль db.js виступає як Data Access Layer і надає функції для кожної з трьох таблиць.

# РОЗДІЛ 2. СТРУКТУРА МЕНЮ

Навігація в додатку реалізована через два елементи: шапку (header) і горизонтальну панель категорій (nav.categories). Обидва елементи описані у файлі public/index.html.

Шапка містить: логотип «NewsHub», поле пошуку з кнопкою, випадаючий список вибору мови (#lang-select із варіантами EN та UA) і кнопку перемикання теми (#theme-toggle). Шапка зафіксована вгорі сторінки через position: sticky, тому залишається видимою при прокручуванні.

Панель категорій містить сім фіксованих вкладок: General, Technology, Sports, Business, Science, Health, Bookmarks. Кожна кнопка має атрибут data-category, який передається у запит до сервера як параметр category. Активна вкладка отримує клас active. Після фіксованих кнопок розміщено блок #custom-tabs, куди JavaScript динамічно додає користувацькі категорії, і кнопка «+» (#add-cat-btn) для їх створення.



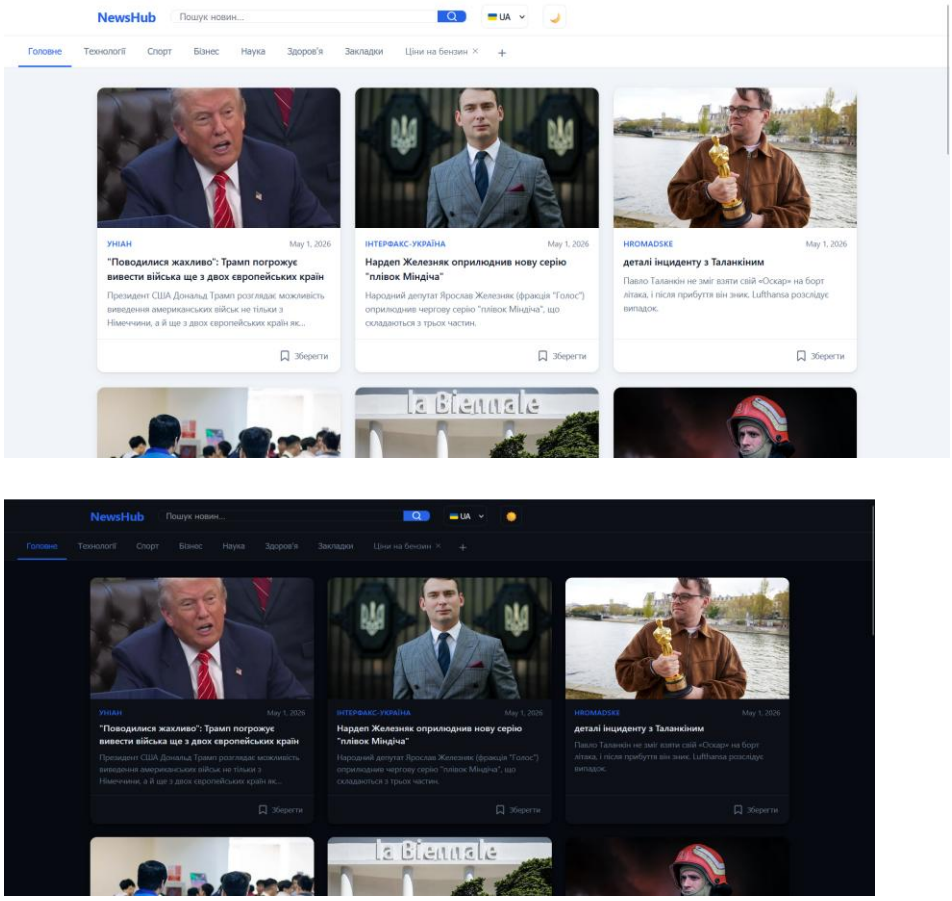

[Рисунок 2.1 – Структура навігації додатку]

При натисканні «+» поле введення з'являється безпосередньо в рядку панелі. Введена назва відправляється через POST /api/categories і після успішної відповіді сервера нова вкладка з'являється в інтерфейсі. Видалення виконується через кнопку-хрестик, яка робить запит DELETE /api/categories/:id.

# РОЗДІЛ 3. РОЗМІЩЕННЯ ТА СТИЛІЗАЦІЯ ІНТЕРФЕЙСУ

Стилізація виконана у файлі public/css/style.css. Для підтримки темної і світлої тем використовуються CSS-змінні (custom properties).

CSS власні властивості (custom properties) це сутності, що містять конкретні значення для повторного використання у документі; оголошуються через нотацію -- і використовуються через функцію var() [6]. У селекторі :root оголошено світлу палітру: --bg: #f1f5f9, --surface: #ffffff, --text: #0f172a, --accent: #2563eb та інші. Для темної теми у селекторі [data-theme="dark"] переозначено --bg: #060910, --surface: #0d1117, --text: #e2e8f0. Перемикання відбувається через зміну атрибута data-theme на елементі html браузер автоматично застосовує відповідний набір змінних, що дає миттєву зміну теми без перезавантаження.







[Рисунок 3.1 – Світла тема (ліворуч) та темна тема (праворуч)]

Картки новин розміщені у CSS Grid: .news-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(300px, 1fr)); gap: 20px }. Така сітка автоматично підлаштовується під ширину екрана  на великих моніторах відображається 3–4 картки в рядку, на мобільних 1. Адаптивність забезпечена без медіа-запитів через властивість auto-fill.

Кожна картка (.card) має тінь var(--shadow), заокруглення var(--card-radius: 12px) і зображення з фіксованою висотою 180px з object-fit: cover, щоб зберігати пропорції незалежно від оригінального розміру картинки. Для скелетон-анімації при завантаженні використовується CSS-анімація skeleton-shimmer, яка імітує мерехтіння контенту.

# РОЗДІЛ 4. МОДЕЛІ ДАНИХ

Для зберігання персональних даних користувача реалізовано три моделі даних, кожна з яких відповідає окремій таблиці SQLite. Всі CREATE TABLE описані у файлі db.js на рядках 17–50.

**Таблиця 4.1 – Структура таблиць бази даних**

| Таблиця | Поля | Призначення |
|---|---|---|
| bookmarks | url (PK), title, description, urlToImage, publishedAt, source, notes, is_read | Збережені статті |
| categories | id (PK AUTO), name (UNIQUE), created_at | Користувацькі категорії |
| search_history | id (PK AUTO), query, searched_at | Історія пошуку |

Модель закладок (bookmarks) використовує URL статті як первинний ключ замість автоінкрементного ID. Це дозволяє уникнути дублікатів при повторному збереженні однієї статті — INSERT OR IGNORE просто ігнорує такий запит. Поля notes та is_read дозволяють додавати нотатки до збережених матеріалів і позначати їх як прочитані.

Модель категорій (categories) зберігає лише назву з обмеженням UNIQUE — це гарантує відсутність однакових вкладок. Поле created_at заповнюється автоматично через DEFAULT (datetime('now')).

Модель історії пошуку (search_history) не прив'язана до інших таблиць. Обмеження на 50 записів підтримується програмно у функції addSearchHistory(): після кожного INSERT виконується DELETE для записів, яких немає серед 50 останніх за id.

# РОЗДІЛ 5. МЕХАНІЗМИ ОТРИМАННЯ ТА ОНОВЛЕННЯ ДАНИХ

Уся взаємодія клієнта з сервером реалізована через Fetch API у файлі public/js/app.js. Fetch API це інтерфейс для виконання HTTP-запитів у браузері, який повертає Promise [7]. Клієнт не має прямого доступу до бази даних або зовнішніх API все проходить через серверний проксі.

Завантаження новин відбувається у функції fetchNews(). Перед запитом функція showSkeletons(9) вставляє у сітку 9 карток-заглушок, щоб уникнути порожнього екрана. Після отримання відповіді скелетони замінюються реальними картками через функцію renderCards(articles).

Стан застосунку зберігається у чотирьох змінних: currentCategory, currentQuery, currentLang і bookmarkedUrls (Set з URL збережених статей). Set обрано замість масиву для того, щоб перевірка «чи збережена стаття» виконувалася за O(1)  це важливо, оскільки перевірка відбувається для кожної картки при рендерингу.

Збереження закладки виконується у функції toggleBookmark(): якщо URL статті є у bookmarkedUrls виконується DELETE /api/bookmarks/:url і URL видаляється з Set; якщо ні виконується POST /api/bookmarks з тілом запиту (title, description, url, urlToImage, publishedAt, source) і URL додається до Set. Кнопка закладки оновлює свій стиль одразу після успішної відповіді без перерендерингу всієї сітки.

Нотатки зберігаються автоматично при втраті фокусу на textarea через обробник blur  окрема кнопка «зберегти» не потрібна. Статус is_read оновлюється при кліку на кнопку через PUT /api/bookmarks/:url з тілом { is_read: 1} або { is_read: 0}.

# РОЗДІЛ 6. БАЗА ДАНИХ

Як СКБД обрано SQLite через бібліотеку sql.js це SQL-бібліотека для JavaScript, що містить версію SQLite, скомпільовану для вебу [3]. SQLite це вбудована реляційна база даних, яка зберігає всі дані в одному файлі на диску [8]. Перевага такого підходу полягає у відсутності нативних залежностей: бібліотека встановлюється як звичайний npm-пакет без компіляції під конкретну ОС. База даних зберігається у файлі bookmarks.db у кореневій директорії проєкту.

Ініціалізація відбувається у функції getDb(). При першому виклику вона завантажує бінарний файл бази через fs.readFileSync() або створює нову порожню базу, виконує CREATE TABLE IF NOT EXISTS для всіх трьох таблиць і перевіряє наявність полів notes та is_read через ALTER TABLE в блоках try/catch. Це реалізація простих міграцій: якщо колонки вже існують, SQLite поверне помилку, яка буде проігнорована.

Після кожної операції запису викликається функція persist() (рядок 57), яка виконує db.export() отримує бінарне представлення бази і записує його у файл через fs.writeFileSync(). Це потрібно, оскільки sql.js тримає базу в пам'яті, а не на диску безпосередньо.

Усі SQL-операції виконуються через параметризовані запити з плейсхолдерами (?), що унеможливлює SQL-ін'єкції.

# РОЗДІЛ 7. ІНТЕГРАЦІЯ СТОРОННІХ СЕРВІСІВ

Для отримання новин інтегровано два зовнішніх сервіси: NewsAPI для англомовного контенту та GNews для україномовного. Обидва сервіси підключені на серверній стороні ключі зберігаються у файлі .env і не потрапляють у клієнтський код.

NewsAPI (newsapi.org) простий HTTP REST API для пошуку та отримання статей з понад 150 000 джерел по всьому світу [4]. Використовуються два ендпоінти: /v2/top-headlines для отримання провідних новин за категоріями та країною, і /v2/everything для пошуку за ключовим словом з сортуванням за датою публікації. Безкоштовний тариф дозволяє 100 запитів на добу і потребує, щоб запити виконувалися з сервера, а не з браузера.

GNews (gnews.io) REST API сервіс для пошуку статей з понад 80 000 джерел у 41 мові по всьому світу [5]. Обрано його за підтримку параметру lang=uk для отримання україномовних новин, якої немає у NewsAPI. Використовуються ендпоінти /api/v4/top-headlines для провідних новин за категоріями та /api/v4/search для пошуку. GNews повертає поле image замість urlToImage, тому у routes/news.js реалізована функція normalizeGNews(), яка приводить відповідь до єдиного формату:

function normalizeGNews(a) { return { title: a.title, description: a.description, url: a.url, urlToImage: a.image, publishedAt: a.publishedAt, source: { name: a.source?.name || '' } }; }

Вибір між сервісами відбувається у головному обробнику GET /api/news залежно від параметру lang: якщо lang === 'uk' — запит іде до GNews, інакше — до NewsAPI. Це дозволяє клієнту перемикати мову без зміни будь-якої іншої логіки.

# РОЗДІЛ 8. СЕРВЕРНА АРХІТЕКТУРА

Node.js це асинхронне подійно-кероване середовище виконання JavaScript, призначене для побудови масштабованих мережевих застосунків [1]. Event loop дозволяє виконувати неблокуючі I/O-операції незважаючи на те, що за замовчуванням використовується один JavaScript-потік [1]. Це підходить для агрегатора новин, де сервер більшу частину часу очікує відповіді від зовнішніх API.

Express мінімальний і гнучкий веб-фреймворк для Node.js, що надає набір функцій для побудови веб-застосунків [2]. Express-застосунок по суті є серією викликів middleware-функцій [2]. Точка входу файл server.js: він підключає express.json() для парсингу тіла запиту, express.static('public') для роздачі статичних файлів і три роутери.

Роутери підключені до трьох шляхів: /api/news, /api/bookmarks, /api/categories. Кожен роутер окремий файл у теці routes/. Такий поділ тримає кожен модуль компактним і зосередженим на одній сутності.

**Таблиця 8.1 – REST API додатку NewsHub**

| Метод | Шлях | Роутер | Опис |
|---|---|---|---|
| GET | /api/news | news.js | Отримати новини (параметри: category, q, lang) |
| GET | /api/news/search-history | news.js | Отримати історію пошуку (останні 20) |
| DELETE | /api/news/search-history | news.js | Очистити всю історію пошуку |
| DELETE | /api/news/search-history/:id | news.js | Видалити один запис з історії |
| GET | /api/bookmarks | bookmarks.js | Отримати всі збережені закладки |

| Метод | Шлях | Роутер | Опис |
|---|---|---|---|
| POST | /api/bookmarks | bookmarks.js | Зберегти нову закладку |
| PUT | /api/bookmarks/:url | bookmarks.js | Оновити нотатку або статус is_read |
| DELETE | /api/bookmarks/:url | bookmarks.js | Видалити закладку |
| GET | /api/categories | categories.js | Отримати всі категорії |
| POST | /api/categories | categories.js | Створити нову категорію |
| DELETE | /api/categories/:id | categories.js | Видалити категорію |

Сервер слухає порт з процесної змінної PORT або 3000 за замовчуванням. Обробник події error на об'єкті server окремо перехоплює код EADDRINUSE і виводить підказку зі командою PowerShell для звільнення порту це зручно при розробці, коли порт залишається зайнятим після неакуратної зупинки.

Усі API-ключі зберігаються у файлі .env і завантажуються на початку server.js через бібліотеку dotenv [9]. Файл .env не потрапляє у репозиторій завдяки запису в .gitignore.

# РОЗДІЛ 9. ОБРОБКА ЗАПИТІВ ТА ВЗАЄМОДІЯ З БАЗОЮ ДАНИХ НА СЕРВЕРІ

Взаємодія з базою даних відбувається виключно через функції, експортовані з db.js. Роутери імпортують тільки ті функції, які їм потрібні, і не виконують SQL-запити напряму  це забезпечує чіткий поділ відповідальностей.

**Таблиця 9.1 – Функції бази даних (db.js)**

| Функція | SQL-запит |
|---|---|
| getAllBookmarks() | SELECT * FROM bookmarks ORDER BY publishedAt DESC |
| addBookmark(article) | INSERT OR IGNORE INTO bookmarks (url, title, …) |
| updateBookmark(url, fields) | UPDATE bookmarks SET notes/is_read WHERE url = ? |
| removeBookmark(url) | DELETE FROM bookmarks WHERE url = ? |
| getAllCategories() | SELECT * FROM categories ORDER BY id ASC |
| addCategory(name) | INSERT OR IGNORE INTO categories (name) VALUES (?) |
| deleteCategory(id) | DELETE FROM categories WHERE id = ? |
| addSearchHistory(query) | INSERT INTO search_history; DELETE надлишкових > 50 |
| getSearchHistory() | SELECT * FROM search_history ORDER BY id DESC LIMIT 20 |
| deleteSearchHistoryItem(id) | DELETE FROM search_history WHERE id = ? |
| clearSearchHistory() | DELETE FROM search_history |

Роутер bookmarks.js при обробці PUT /:url декодує URL з path-параметра через decodeURIComponent, оскільки URL містить спеціальні символи. Функція updateBookmark() у db.js перед виконанням запиту

фільтрує ключі через білий список allowed = ['notes', 'is_read'] — це захист від спроби оновити первинний ключ або інші системні поля.

Роутер news.js після успішного виклику зовнішнього API автоматично записує пошуковий запит в історію через addSearchHistory(q.trim()), якщо параметр q присутній у запиті. Таким чином клієнту не потрібно робити окремий запит для збереження в історію.

## РОЗДІЛ 10. ТЕСТУВАННЯ ТА НАЛАШТУВАННЯ ВЕБ-ДОДАТКУ

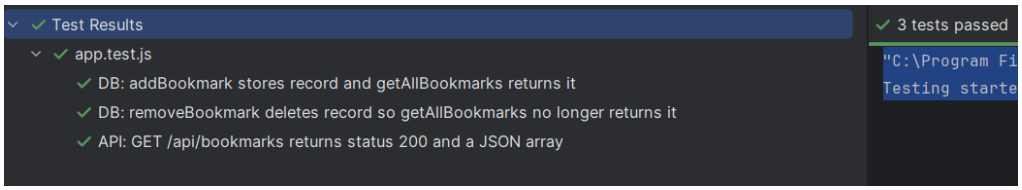
Для перевірки коректності роботи застосунку написано три автоматичних тести у файлі tests/app.test.js з використанням вбудованого модуля node:test [10], який доступний починаючи з Node.js 18 без встановлення додаткових залежностей.

Перший тест перевіряє функцію addBookmark(): додає тестову закладку, викликає getAllBookmarks() і перевіряє, що запис присутній у базі з правильними полями title та source. Після тесту закладка видаляється через removeBookmark().

Другий тест перевіряє функцію removeBookmark(): додає закладку, видаляє її і перевіряє через getAllBookmarks(), що запис відсутній у результаті.

Третій тест перевіряє HTTP-рівень: піднімає Express-сервер на довільному порту (port 0  ОС сама призначає вільний порт), виконує реальний GET-запит на /api/bookmarks і перевіряє, що відповідь має статус 200 і тіло є масивом JSON. Після завершення сервер закривається.

Запуск тестів: npm test (скрипт визначений у package.json як node --test tests/app.test.js). Усі три тести проходять успішно.




[Рисунок 10.1 – Результат виконання тестів у терміналі]

Для запуску застосунку необхідно: встановити залежності командою npm install, створити файл .env з ключами NEWS_API_KEY та GNEWS_API_KEY, запустити сервер командою npm start. Після цього додаток доступний у браузері за адресою http://localhost:3000.

# ВИГЛЯД СТОРІНОК ВЕБ-САЙТУ

У цьому розділі наведено скріншоти основних екранів додатку NewsHub.

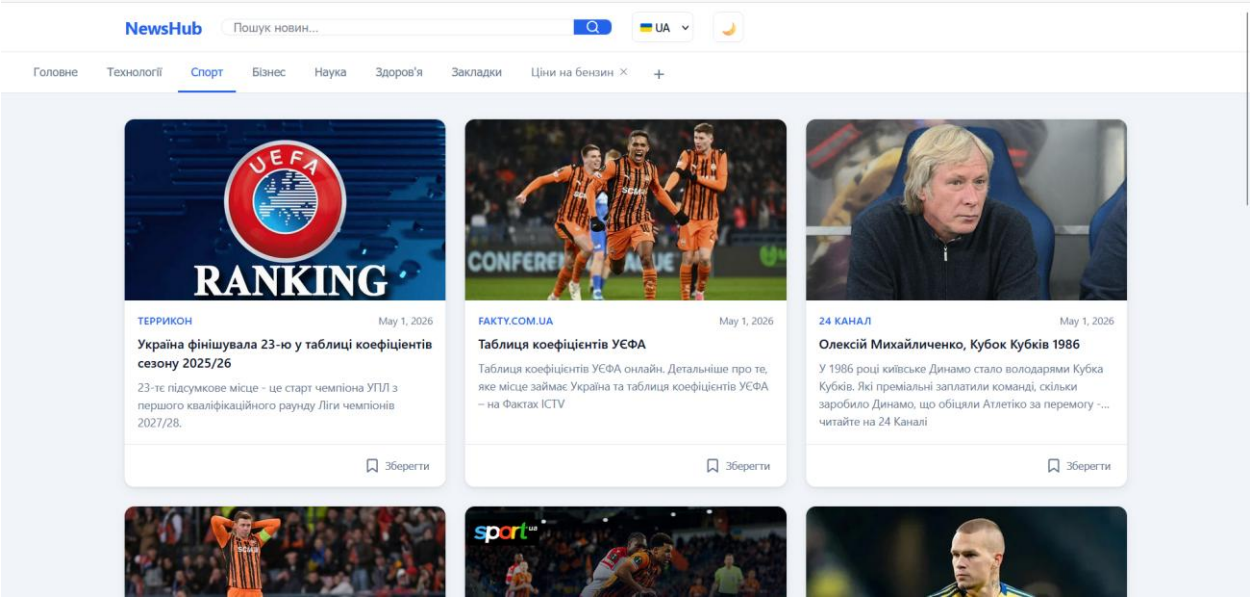



Рис. 1 – Головна сторінка (світла тема, категорія General)

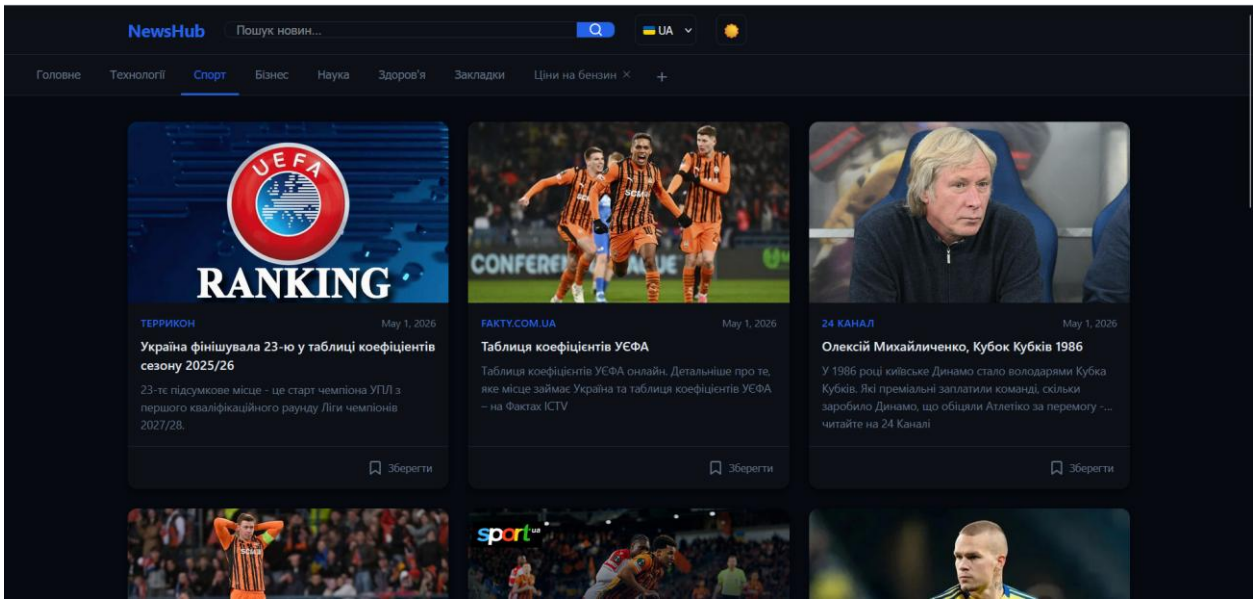



Рис. 2 – Головна сторінка (темна тема)

Рис. 3 – Пошук новин та список історії пошуку

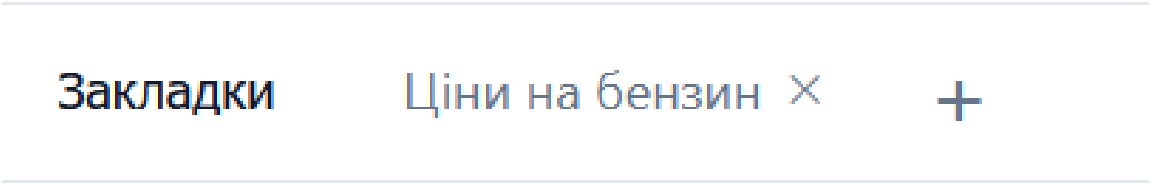



Рис. 4 – Вкладка «Bookmarks»

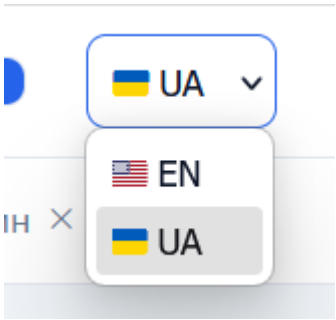



Рис. 5 – Україномовний режим (GNews, темна тема)

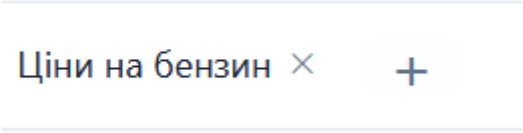



Рис. 6 – Додавання користувацької категорії

# ВИСНОВКИ

У ході курсової роботи розроблено веб-додаток «NewsHub» агрегатор новин з підтримкою двох мов та двох тем оформлення. Усі поставлені завдання виконано.

Спроєктовано діаграми компонентів, взаємодії та класів, які відображають архітектуру системи. Реалізовано зручну навігацію через горизонтальну панель вкладок із можливістю додавати та видаляти власні категорії.

Розроблено серверний REST API на Node.js / Express з 11 ендпоінтами, розподіленими між трьома роутерами. Спроєктовано базу даних SQLite з трьома таблицями (bookmarks, categories, search_history) та реалізовано повний CRUD для кожної сутності через функції db.js.

Інтегровано два зовнішніх сервіси  NewsAPI для англомовних новин та GNews для україномовних. Клієнтська частина реалізована на vanilla JavaScript без фреймворків, стилізація  на чистому CSS з використанням CSS-змінних для підтримки тем.

Написано три автоматичних тести з використанням node:test, які перевіряють роботу бази даних та HTTP-рівня API. Усі тести проходять успішно.

Отримано практичний досвід роботи з Node.js, Express, SQL, Fetch API, CSS custom properties та інтеграцією зовнішніх REST API у реальному проєкті.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

**СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ**

1. Node.js About Node.js.

URL: https://nodejs.org/about (дата звернення: 18.04.2026).

2. Express.js  Express: Fast, unopinionated, minimalist web framework for Node.js. URL: https://expressjs.com/ (дата звернення: 18.04.2026).

3. sql.js SQL library for JavaScript containing a version of SQLite compiled for the web.

URL: https://sql.js.org/ (дата звернення: 19.04.2026).

4. NewsAPI Documentation.

URL: https://newsapi.org/docs (дата звернення: 20.04.2026).

5. GNews API Documentation.

URL: https://gnews.io/docs/ (дата звернення: 20.04.2026).

6. MDN Web Docs Using CSS custom properties (variables).

URL: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Cascading_variables/Using_custom_properties(дата звернення: 21.04.2026).

7. MDN Web Docs Fetch API.

URL:https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API(дата звернення: 21.04.2026).

8. SQLite About SQLite.

URL: https://www.sqlite.org/about.html (дата звернення: 19.04.2026).

9. dotenv npm package.

URL: https://www.npmjs.com/package/dotenv (дата звернення: 20.04.2026).

10. Node.js  Test runner.

URL: https://nodejs.org/api/test.html (дата звернення: 22.04.2026).

## ДОДАТОК А

Фрагмент коду, файл server.js

```
require('dotenv').config();
const express = require('express');

const app  = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
app.use(express.static('public'));

app.use('/api/news',        require('./routes/news'));
app.use('/api/bookmarks',   require('./routes/bookmarks'));
app.use('/api/categories',  require('./routes/categories'));

const server = app.listen(PORT, () => console.log(`Server running at
http://localhost:${PORT}`));

server.on('error', err => {
  if (err.code === 'EADDRINUSE') {
    console.error(`\nPort ${PORT} is already in use.`);
    console.error(`Run this in PowerShell to free it:`);
    console.error(`  Stop-Process -Id (Get-NetTCPConnection -LocalPort
```

```
${PORT}).OwningProcess -Force\n`);
  } else {
    console.error(err);
  }
  process.exit(1);
});
```

## ДОДАТОК Б

Фрагмент коду, файл db.js

```
const initSqlJs = require('sql.js');
const fs = require('fs');
const path = require('path');

const DB_PATH = path.join(__dirname, 'bookmarks.db');

let db;

async function getDb() {
  if (db) return db;

  const SQL = await initSqlJs();
  db = fs.existsSync(DB_PATH)
    ? new SQL.Database(fs.readFileSync(DB_PATH))
    : new SQL.Database();

  db.run(`
    CREATE TABLE IF NOT EXISTS bookmarks (
      url         TEXT PRIMARY KEY,
      title       TEXT,
      description TEXT,
      urlToImage  TEXT,
      publishedAt TEXT,
      source      TEXT,
```

```
    notes       TEXT    DEFAULT '',
    is_read     INTEGER DEFAULT 0
  )
`);

// Migration: add columns to existing DBs that predate this schema
try { db.run("ALTER TABLE bookmarks ADD COLUMN notes TEXT
DEFAULT ''"); } catch (_) {}
try { db.run('ALTER TABLE bookmarks ADD COLUMN is_read INTEGER
DEFAULT 0'); } catch (_) {}

// Stores user-defined category tabs (moved from localStorage to DB)
db.run(`
  CREATE TABLE IF NOT EXISTS categories (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    name       TEXT UNIQUE NOT NULL,
    created_at TEXT DEFAULT (datetime('now'))
  )
`);

// Keeps the last 50 searches for history/analytics
db.run(`
  CREATE TABLE IF NOT EXISTS search_history (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    query       TEXT NOT NULL,
    searched_at TEXT DEFAULT (datetime('now'))
  )
`);

return db;
}

function persist() {
  fs.writeFileSync(DB_PATH, Buffer.from(db.export()));
}

//  Bookmarks
async function getAllBookmarks() {
  const d = await getDb();
```

```
  const stmt = d.prepare('SELECT * FROM bookmarks ORDER BY publishedAt
DESC');
  const rows = [];
  while (stmt.step()) rows.push(stmt.getAsObject());
  stmt.free();
  return rows;
}

async function addBookmark(article) {
  const d = await getDb();
  d.run(
    `INSERT OR IGNORE INTO bookmarks (url, title, description, urlToImage,
publishedAt, source)
     VALUES (?, ?, ?, ?, ?, ?)`,
    [article.url, article.title, article.description, article.urlToImage,
article.publishedAt, article.source]
  );
  persist();
}

async function updateBookmark(url, fields) {
  const d = await getDb();
  // Only allow updating safe fields — notes and is_read
  const allowed = ['notes', 'is_read'];
  const keys = Object.keys(fields).filter(k => allowed.includes(k));
  if (!keys.length) return;
  const set = keys.map(k => `${k} = ?`).join(', ');
  d.run(`UPDATE bookmarks SET ${set} WHERE url = ?`, [...keys.map(k =>
fields[k]), url]);
  persist();
}

async function removeBookmark(url) {
  const d = await getDb();
  d.run('DELETE FROM bookmarks WHERE url = ?', [url]);
  persist();
}

//  Categories
```

```
async function getAllCategories() {
  const d = await getDb();
  const stmt = d.prepare('SELECT * FROM categories ORDER BY id ASC');
  const rows = [];
  while (stmt.step()) rows.push(stmt.getAsObject());
  stmt.free();
  return rows;
}

async function addCategory(name) {
  const d = await getDb();
  d.run('INSERT OR IGNORE INTO categories (name) VALUES (?)', [name]);
  persist();
  const stmt = d.prepare('SELECT * FROM categories WHERE name = ?');
  stmt.bind([name]);
  stmt.step();
  const row = stmt.getAsObject();
  stmt.free();
  return row;
}

async function deleteCategory(id) {
  const d = await getDb();
  d.run('DELETE FROM categories WHERE id = ?', [id]);
  persist();
}

// Search history
async function addSearchHistory(query) {
  const d = await getDb();
  d.run('INSERT INTO search_history (query) VALUES (?)', [query]);
  // Keep only the last 50 entries
  d.run(`DELETE FROM search_history WHERE id NOT IN
    (SELECT id FROM search_history ORDER BY id DESC LIMIT 50)`);
  persist();
}

async function getSearchHistory() {
  const d = await getDb();
```

```
  const stmt = d.prepare('SELECT * FROM search_history ORDER BY id DESC
LIMIT 20');
  const rows = [];
  while (stmt.step()) rows.push(stmt.getAsObject());
  stmt.free();
  return rows;
}

async function deleteSearchHistoryItem(id) {
  const d = await getDb();
  d.run('DELETE FROM search_history WHERE id = ?', [id]);
  persist();
}

async function clearSearchHistory() {
  const d = await getDb();
  d.run('DELETE FROM search_history');
  persist();
}

module.exports = {
  getAllBookmarks, addBookmark, updateBookmark, removeBookmark,
  getAllCategories, addCategory, deleteCategory,
  addSearchHistory, getSearchHistory, deleteSearchHistoryItem,
clearSearchHistory,
};
```